PPE Detection System: Architecture Document

Factory CCTV-based detection of missing personal protective equipment (PPE), built on a Python inference/API stack and a Next.js dashboard.

1. Overview

Purpose. Analyze live CCTV streams across the factory, detect people who are missing required PPE (helmet, vest, gloves, goggles, boots), and alert supervisors in near real time. Every violation is stored with evidence for review and reporting.

Design principles

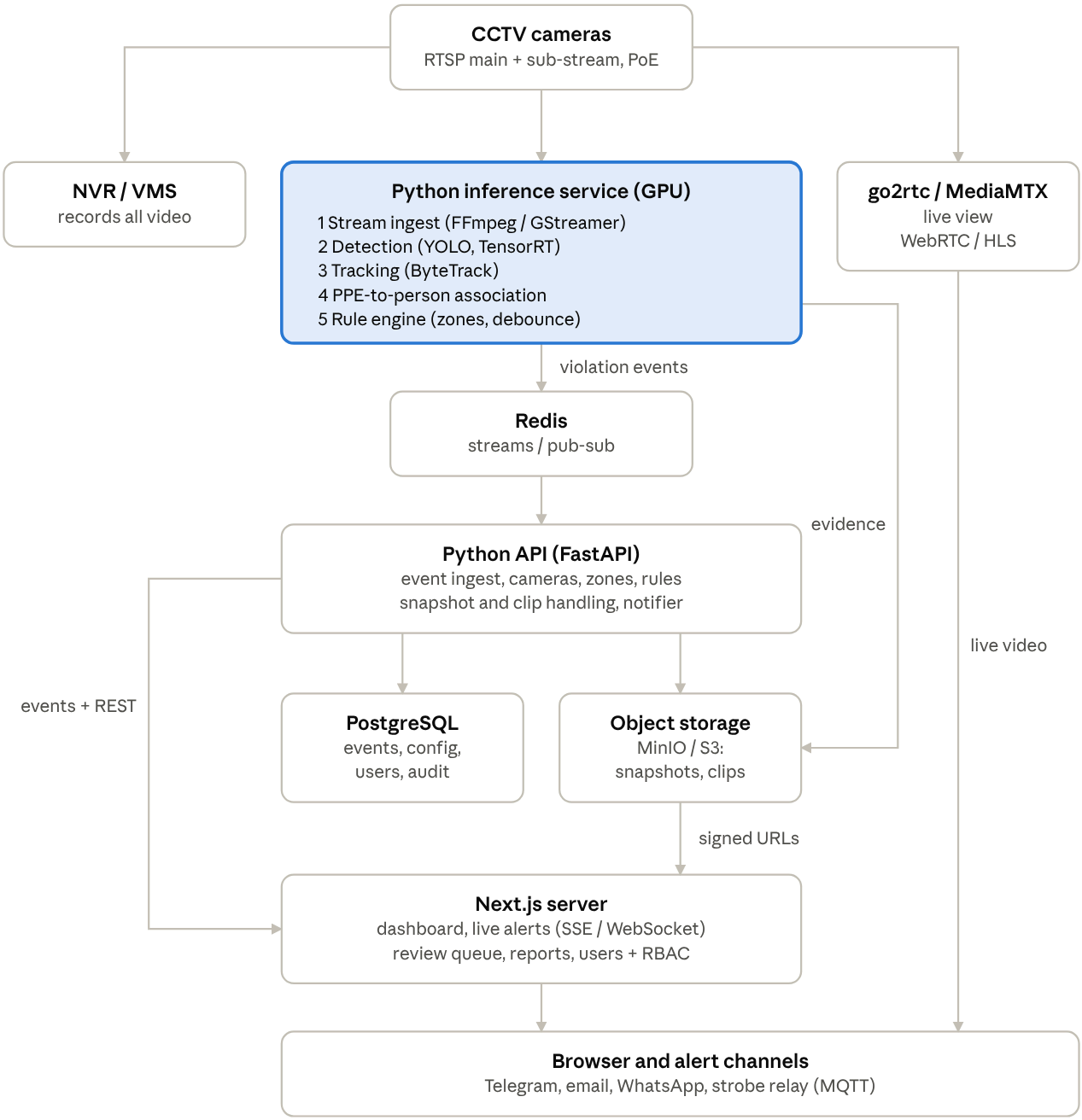
- **On-prem video.** Video stays inside the factory network. Only events and snapshots are stored.
- **Separation of concerns.** Python does vision and rules. Next.js does UI, auth, and reporting. The NVR/VMS does continuous recording.
- **Event-driven.** Components communicate through events and queues so each can scale or fail independently.
- **Config over code.** Zones, required PPE, thresholds, and schedules are data in the database, not hard-coded.
- **Human in the loop.** Reviewers confirm or reject violations, and rejected ones feed model retraining.

Assumptions (to be confirmed)

Item	Assumption
Cameras	IP cameras with RTSP/ONVIF, PoE
Camera count	20 to 50 for the first full deployment
Location	One factory site, possibly several buildings
Latency target	Alert within 3 to 5 seconds of a confirmed violation
Retention	Snapshots and clips 90 days, continuous video per NVR policy

2. Logical Architecture

Video becomes events at one GPU service; everything else handles events



system architecture · 10 components

Cameras feed the inference service, which turns video into violation events. Redis passes them to the API, which stores them and feeds the dashboard and alerts.

3. Physical Architecture and Hardware

3.1 Cameras

Aspect	Specification
Type	IP camera, RTSP + ONVIF, PoE, H.264/H.265
Resolution	2MP for most areas, 4MP for wide halls
Mounting	3 to 4 m high, angled down 30 to 45 degrees
Subject size	At least about 150 px tall per person
Lighting	WDR at doors and gates, IR for dim areas
Streams	Main stream for inference or recording, sub-stream for live dashboard

3.2 Network

- PoE access switches per zone, connected by fiber uplinks to a core switch.
- Cameras on a dedicated VLAN, isolated from the office network, with no internet access.
- Bandwidth estimate: 2 to 4 Mbps per 1080p H.264 camera, so 50 cameras need about 100 to 200 Mbps.
- Switches and servers on UPS power.

3.3 Compute

Role	Hardware	Notes
Inference server	On-prem server with RTX A4000 or L4 class GPU	Roughly 15 to 30 cameras per GPU at 5 to 10 FPS each. Benchmark with your model before buying.
App server	8+ vCPU, 16+ GB RAM	Runs FastAPI, Next.js, PostgreSQL, Redis. Can be a VM.
Edge box (optional)	Jetson Orin per zone	For distant buildings. Sends events only to the central server.

Role	Hardware	Notes
Storage	NAS or object storage	MinIO for snapshots/clips. NVR has its own disks.

3.4 Optional alert hardware

Strobe lights or buzzers at gates and hazard zones, triggered by a relay or MQTT message.

4. Component Design

4.1 Stream ingest

- One reader per camera using FFmpeg or GStreamer with NVDEC hardware decoding.
- Sample frames at 5 to 10 FPS instead of processing the full frame rate.
- Auto-reconnect with exponential backoff, and publish a camera health heartbeat.
- Keep only the latest frame in a small buffer so the pipeline never falls behind (drop old frames, not new ones).

4.2 Detection

- YOLO model (Ultralytics) fine-tuned on factory data.
- Classes: person, helmet, vest, gloves, goggles, boots. Consider "no_helmet" and "no_vest" classes only if testing shows they beat the absence approach.
- Export to TensorRT (FP16) for throughput. Batch frames from several cameras per GPU call.
- Per-class confidence thresholds, tuned per camera if needed.

4.3 Tracking and association

- ByteTrack assigns a stable track ID per person.
- Each PPE detection is matched to a person box by position (for example, a helmet must sit in the upper part of the person box).
- A person's PPE state is smoothed over a time window, so one missed frame does not flip the state.

4.4 Rule engine

- Zones are polygons stored per camera. A person is inside a zone based on the foot point (bottom center of the box).
- Each zone has a required-PPE set, an active schedule, and an optional severity.
- A violation is raised only when the same track ID is missing required PPE for N consecutive seconds, then a cooldown suppresses repeats.
- Ignore rules: tiny boxes, edge-of-frame partial bodies, and masked regions (for example, a screen showing people).

4.5 Evidence capture

- Save a full-frame snapshot with boxes drawn, plus a cropped person image.
- Save a 10 to 20 second clip: a rolling pre-event buffer plus a post-event tail.
- Upload to object storage and attach the keys to the event.

4.6 Python API (FastAPI)

- Receives violation events from Redis (or HTTP), writes them to PostgreSQL, and fans them out.
- Exposes REST endpoints for cameras, zones, rules, violations, and system health.
- Pushes the active configuration to inference workers (poll or pub-sub on change).
- Runs the notifier: Telegram, email, WhatsApp, and MQTT to strobe relays, with retries and a dead-letter queue.

4.7 Next.js server

- Dashboard: live alerts, today's violation count, camera status map, top violation zones.
- Review queue: view evidence, mark confirmed or false alarm, add a comment.
- Reports: by zone, shift, PPE type, and trend over time, with CSV/PDF export.
- Admin: camera onboarding, zone polygon editor drawn over a camera snapshot, PPE rule configuration, users and roles.
- Live alerts through SSE or WebSocket. It does no video processing.

4.8 Live video

- go2rtc or MediaMTX pulls the camera sub-stream and republishes it as WebRTC or HLS.

- The browser never receives raw RTSP, and camera credentials never reach the client.

5. Data Flow

5.1 Violation flow

1. Camera streams RTSP to the inference worker.
2. The worker decodes, detects, tracks, and evaluates rules.
3. A confirmed violation creates an event with camera, zone, track ID, missing items, timestamp, and confidence.
4. The worker saves evidence to object storage and publishes the event to Redis.
5. The Python API consumes the event, writes it to PostgreSQL, and triggers notifications.
6. The API pushes the event to Next.js, which updates open dashboards in real time.
7. A reviewer confirms or rejects it. The result is stored as a label for retraining.

5.2 Configuration flow

1. An admin edits zones or rules in the Next.js UI.
2. Next.js calls the Python API, which validates and saves to PostgreSQL.
3. The API publishes a config-changed message.
4. Inference workers reload their rules without restarting.

6. Data Model (PostgreSQL)

Table	Key fields
cameras	id, name, location, rtsp_url (encrypted), substream_url, status, last_seen
zones	id, camera_id, name, polygon (JSON), active_schedule
ppe_rules	id, zone_id, required_items (array), min_duration_sec, cooldown_sec, severity

Table	Key fields
violations	id, camera_id, zone_id, track_id, missing_items, confidence, occurred_at, snapshot_key, clip_key, status, reviewed_by, reviewed_at
notifications	id, violation_id, channel, status, attempts, sent_at
users	id, name, email, role, active
audit_log	id, user_id, action, entity, entity_id, timestamp
model_versions	id, name, version, deployed_at, metrics

Index violations on (occurred_at), (camera_id, occurred_at), and (zone_id, status). Partition by month if volume grows.

7. Interfaces

7.1 Key REST endpoints (FastAPI)

Method	Path	Purpose
GET	/cameras	List cameras and health
POST	/cameras	Register a camera
GET/PUT	/zones/{id}	Read or update a zone polygon
GET/PUT	/rules/{zone_id}	Required PPE per zone
GET	/violations	Filter by date, zone, status
PATCH	/violations/{id}	Confirm or mark as false alarm
GET	/stats/summary	Dashboard aggregates
GET	/health	Service and worker status

7.2 Event schema (Redis stream)

```
{
  "event_id": "uuid",
  "camera_id": 12,
  "zone_id": 4,
  "track_id": 381,
  "missing": ["helmet", "vest"],
  "confidence": 0.91,
  "occurred_at": "2026-10-23T09:14:05Z",
  "snapshot_key": "violations/2026/10/23/evt_ab12.jpg",
  "clip_key": "violations/2026/10/23/evt_ab12.mp4",
  "model_version": "ppe-yolo-v1.3"
}
```

8. Security and Privacy

Area	Control
Network	Camera VLAN isolated, firewall allows only inference, NVR, and go2rtc hosts
Credentials	RTSP passwords stored encrypted, never exposed to the browser
Auth	Login with role-based access: Admin, Safety Officer, Supervisor, Viewer
Transport	TLS between browser and Next.js, and between internal services where possible
Evidence access	Short-lived signed URLs for snapshots and clips
Audit	Log reviews, config changes, exports, and evidence views
Privacy	Defined retention, worker notification and policy, restricted access to named-person evidence, and masking of non-work areas such as restrooms

9. Observability and Operations

- **Metrics:** per-camera FPS, inference latency, queue depth, GPU utilization and temperature, and stream reconnect counts (Prometheus + Grafana).
- **Logs:** structured logs from all services to a central store (Loki or ELK).
- **Alerts:** camera offline, worker down, GPU overheating, queue backlog, and a sudden drop or spike in detections.
- **Backups:** nightly PostgreSQL dumps and object-storage replication.
- **Deployment:** Docker Compose for the pilot, then Kubernetes or systemd-managed containers for production. GPU containers use the NVIDIA Container Toolkit.
- **Model lifecycle:** versioned models, shadow-mode testing before promotion, and one-command rollback.

10. Scalability and Failure Handling

Scenario	Behavior
Camera drops	Worker reconnects with backoff, dashboard shows offline status, alert after a threshold
Inference overload	Lower per-camera FPS automatically, then drop frames before adding latency
Redis down	Workers buffer events locally and retry
Notification channel fails	Retry, then fall back to another channel, and keep the dead-letter record
GPU failure	Workers on that GPU are rescheduled to a spare, or run at reduced FPS on CPU for critical cameras
Growth beyond one GPU	Add GPUs and shard cameras by group, with each worker owning a camera set
Multiple buildings	Edge boxes run inference locally and send events plus snapshots to the central server

11. Implementation Roadmap

Phase	Duration	Deliverables
0. Survey	1 week	Camera positions, lighting, network check, PPE policy per zone
1. Pilot	3 to 4 weeks	2 to 4 cameras, base model, data collection and labeling, basic dashboard
2. Tuning	2 to 3 weeks	Fine-tuned model, per-camera thresholds, precision and recall report, review workflow
3. Rollout	4 to 8 weeks	Zone-by-zone expansion, notifications, reports, RBAC
4. Hardening	Ongoing	Monitoring, retraining loop, backups, and policy review

Success metrics: recall above 90% and precision above 85% on confirmed violations per camera (adjust to the factory's risk level), false alarms under an agreed number per camera per shift, and alert latency under 5 seconds.

12. Risks and Mitigations

Risk	Mitigation
False alarms erode trust	Duration and cooldown rules, per-camera tuning, human review
Poor camera angles or small subjects	Site survey, reposition or add cameras, use 4MP where needed
Model does not fit factory uniforms or lighting	Collect and label local data from the pilot
Occlusion in crowded areas	Track smoothing, multiple viewpoints, accept some misses
Worker privacy and acceptance	Clear policy, communication, limited access and retention

Risk	Mitigation
GPU sizing wrong	Benchmark the real model and cameras before purchase

13. Open Questions

1. How many cameras, and in how many buildings?
2. Which PPE items are mandatory in which zones?
3. Is there an existing NVR/VMS, and which brand?
4. Which notification channels are preferred (Telegram, email, WhatsApp, strobe)?
5. Required retention period for evidence?
6. Is any integration needed with HR, access control, or the Buildz platform?